

Learning When to Use Each Sorting Algorithm

A machine learning selector for practical runtime behavior

93.1%

runtime gap closed

76.1%

test accuracy

89.6%

zero-regret decisions

How I will defend the work

I start from the decision problem, then show how the final selector was tested.

1 Task and importance

Why one sorting algorithm is not always the practical choice

2 Data and features

Five domains, measured labels, and cheap structural summaries

3 Model and evaluation

XGBoost v5, accuracy, regret, SBS, VBS, and gap closed

4 Evidence and limits

Runtime value, hard cases, strict checks, and what I do not claim

The task is one decision before sorting

- Given one input array, predict which algorithm will finish fastest on that array.



This is not replacing sorting.
It is choosing the better implementation for this input.

Why this matters in practice

| The useful question is not only correctness; it is how much avoidable runtime we leave on the table.

Same input, different cost

A sorted-like array, a random array,
and a duplicated array can reward
different implementation behavior.

Baseline

SBS

one algorithm
for all inputs

Target

VBS

best algorithm
per input

The model is useful if it moves close to VBS
without trying every algorithm.

What previous work gives us

| The idea is connected to algorithm selection, adaptive sorting, and learned performance prediction.

Algorithm selection

Rice formalized choosing a solver from problem features.

My use: sorting algorithm choice as a supervised task.

Adaptive sorting

Timsort and related work exploit runs and existing order in data.

My use: expose these signals as measurable features.

Performance models

ML can predict runtime behavior when features are cheap enough.

My use: evaluate by regret, not only label accuracy.

So the contribution is not a new sorter.
It is a measured selector around existing sorters.

- Build a selector that is cheap enough to run before sorting and close most of the practical runtime gap.

Can simple array structure predict the best sorting algorithm?

Accuracy

Does the label match?

Regret

How costly is a mistake?

Runtime value

How close to VBS?

This is why the result is defended with both model accuracy and runtime regret.

The data was intentionally not from one shape

| The final dataset contains 1.18M arrays from five real-data domains.

1.18M

labeled array instances

F1 telemetry

Stock

Crypto

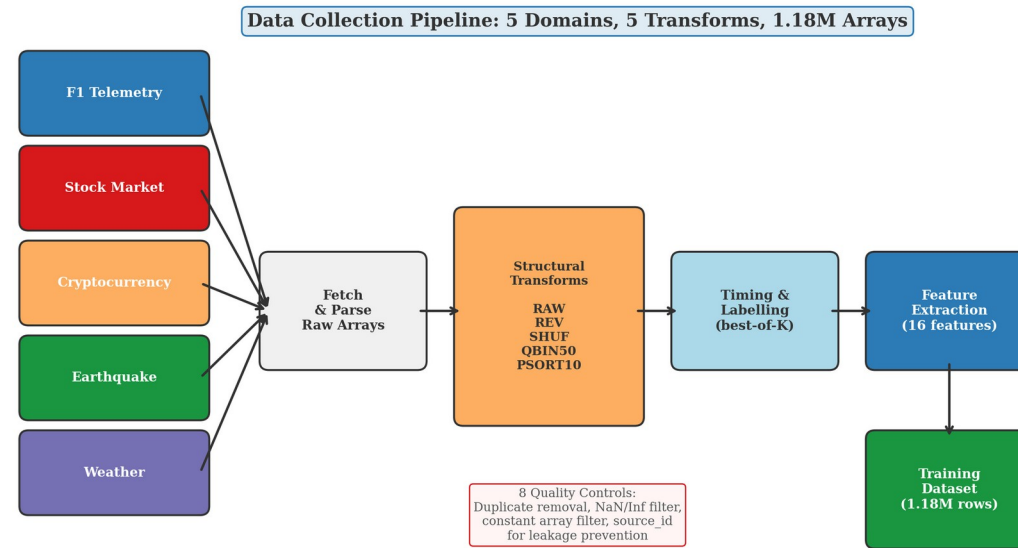
Weather

Earthquake

The point was to make the selector face different numeric patterns, not memorize one dataset family.

Labels came from measured runtime

- Each array was transformed, timed across the portfolio, and labeled by the fastest observed algorithm.



RAW

REV

SHUF

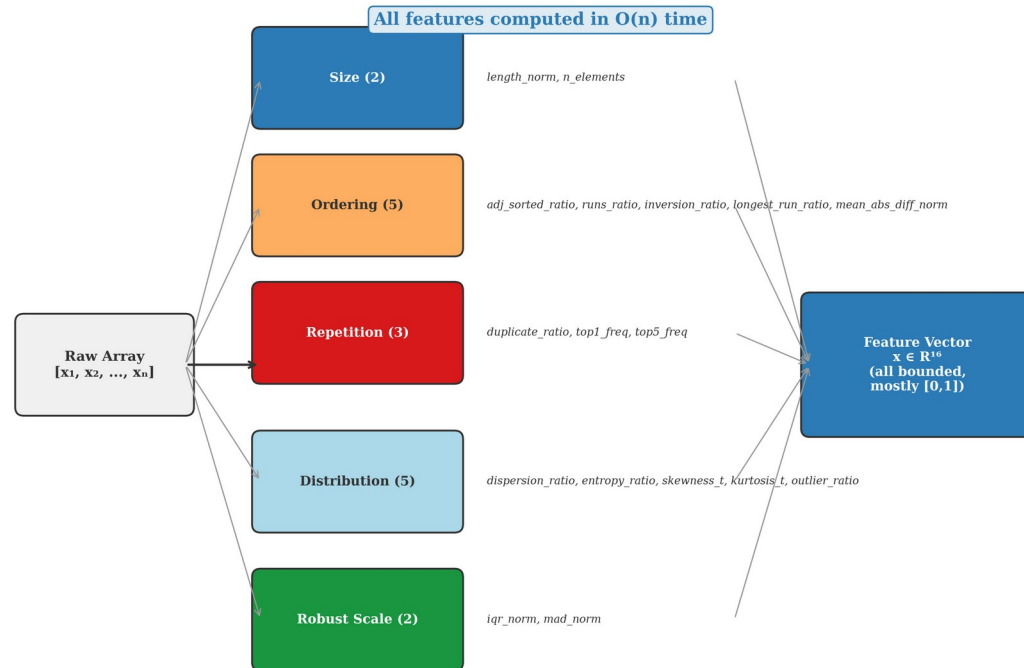
QBIN50

PSORT10

Transforms forced the model to see sorted, reversed, shuffled, quantized, and partially sorted behavior.

Features translate array shape into model input

- The 16 features describe structure that sorting algorithms react to: size, order, duplicates, and local disorder.



Scale

length
range
spread

Shape

inversions
runs
duplicates

The features are not timing features.
They are cheap summaries before sorting.

The selector must be cheaper than the decision it improves

Features are computed from the input structure; the system does not benchmark all algorithms at prediction time.

Allowed before sorting

Read the array once

Count order and duplicate signals

Predict one algorithm class

Not allowed at prediction time

Sort with introsort

Sort with heapsort

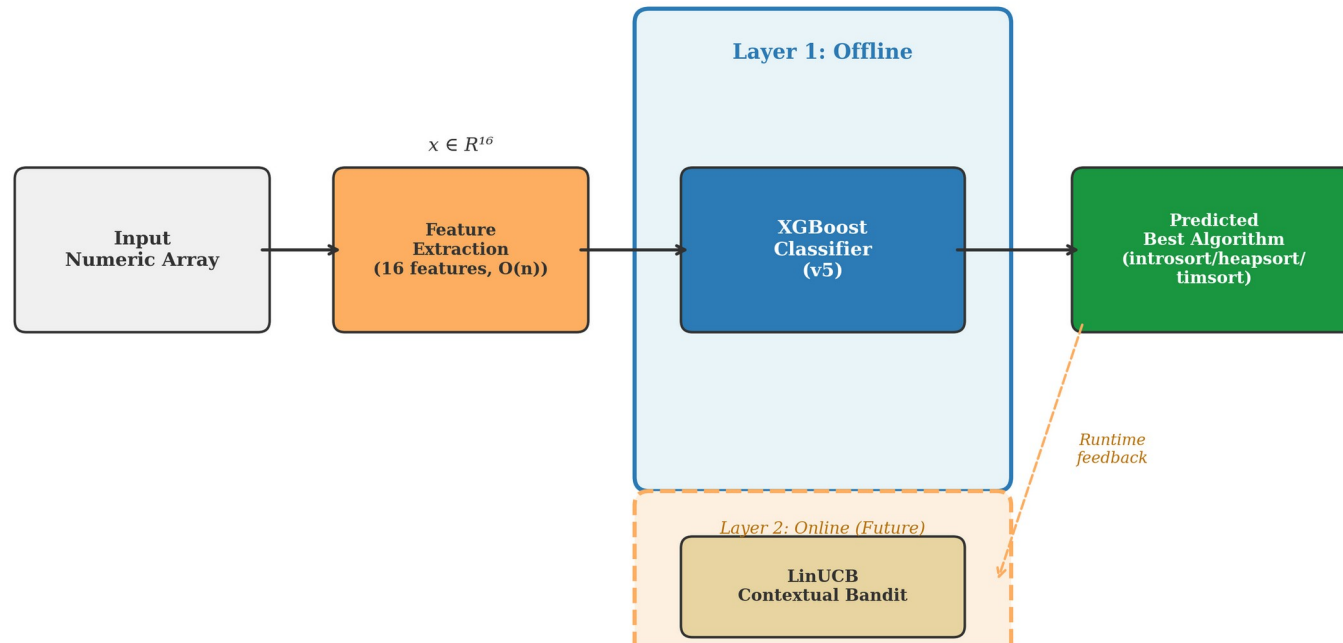
Sort with timsort

Pick after seeing all times

This is the difference between a real selector and an oracle used only for evaluation.

The main system is one general v5 selector

- v5 is trained across all domains and predicts among introsort, heapsort, and timsort.



Array features

XGBoost v5

One algorithm choice

The F1-specific route exists, but it is a se

The portfolio has three real choices

| The model does not rank every possible sorter; it chooses among the three measured implementations.

Intro

general-purpose hybrid
strong default behavior

Heapsort

consistent worst-case bound
often hard to distinguish

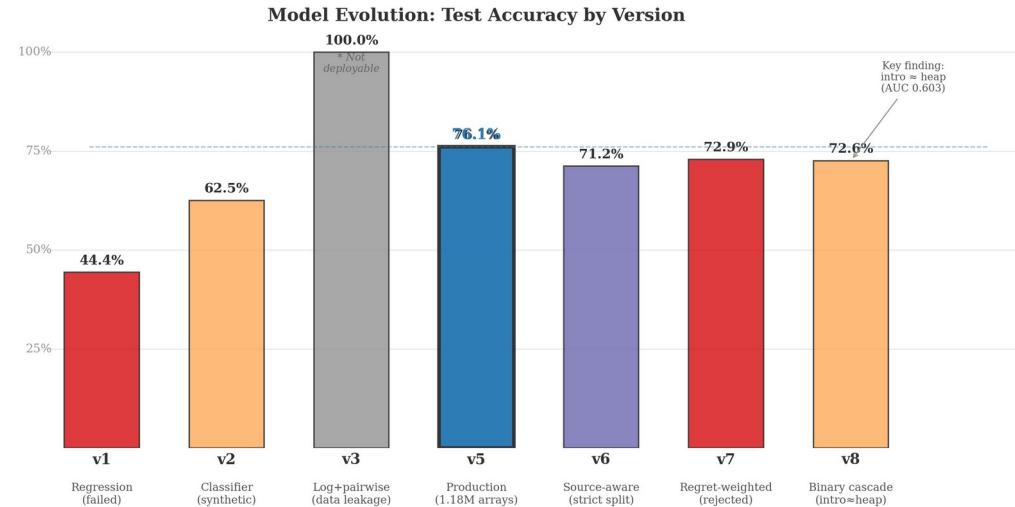
Tim

adaptive to runs
often clear when data has o

The defense is about learning this choice reliably enough to reduce runtime regret.

v5 was chosen after weaker paths were rejected

The research journey matters because it explains why the final model is measured, not accidental.



Regression
not enough

Timing feature
not deployable

v5 classifier
final main model

Strict checks
kept as evidence

Evaluation asks how close we are to the oracle

| Accuracy says whether the class is right; regret says whether the runtime consequence matters.

SBS

single best algorithm
over the test set

VBS

fastest measured algorithm
for each input

Model

choose before sorting
using features only

$$\text{Regret} = \frac{T_{\text{model}} - T_{VBS}}{T_{VBS}}$$

$$\text{Gap Closed} = \frac{T_{SBS} - T_{\text{model}}}{T_{SBS} - T_{VBS}}$$

These formulas make the runtime result defensible, not just the classifier result.

The selector closes most of the runtime gap

The most important result is runtime value: v5 recovers 93.1% of the SBS-to-VBS gap.

93.1%

runtime gap closed

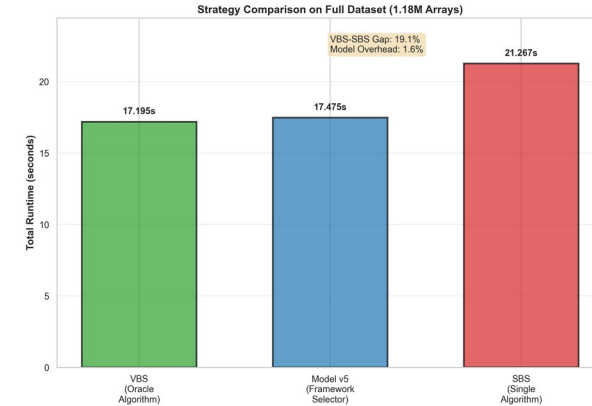
This is the headline because a wrong label is not always an expensive mistake.

76.1%

test accuracy

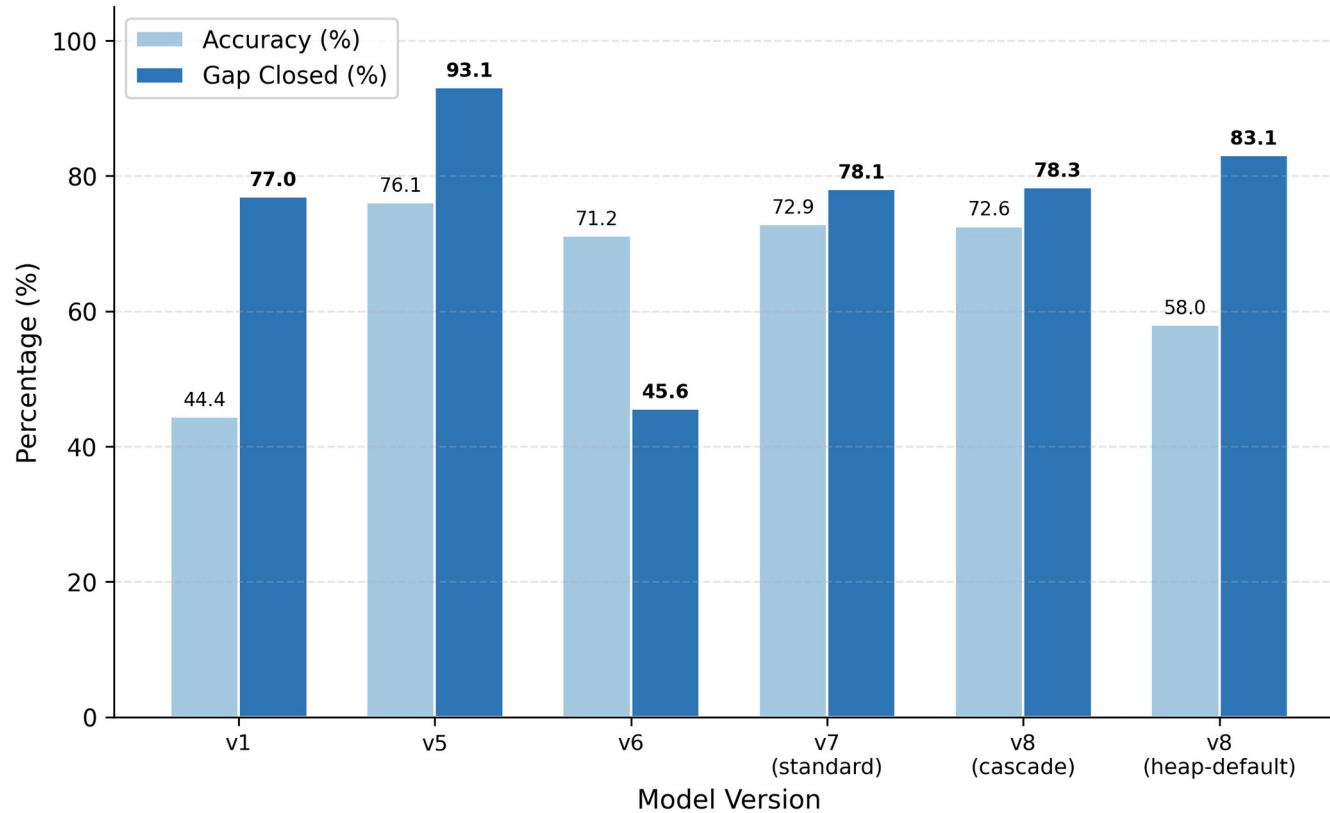
89.6%

zero regret



Accuracy alone does not explain runtime value

Some wrong labels are cheap because two algorithms finish almost the same on that input.



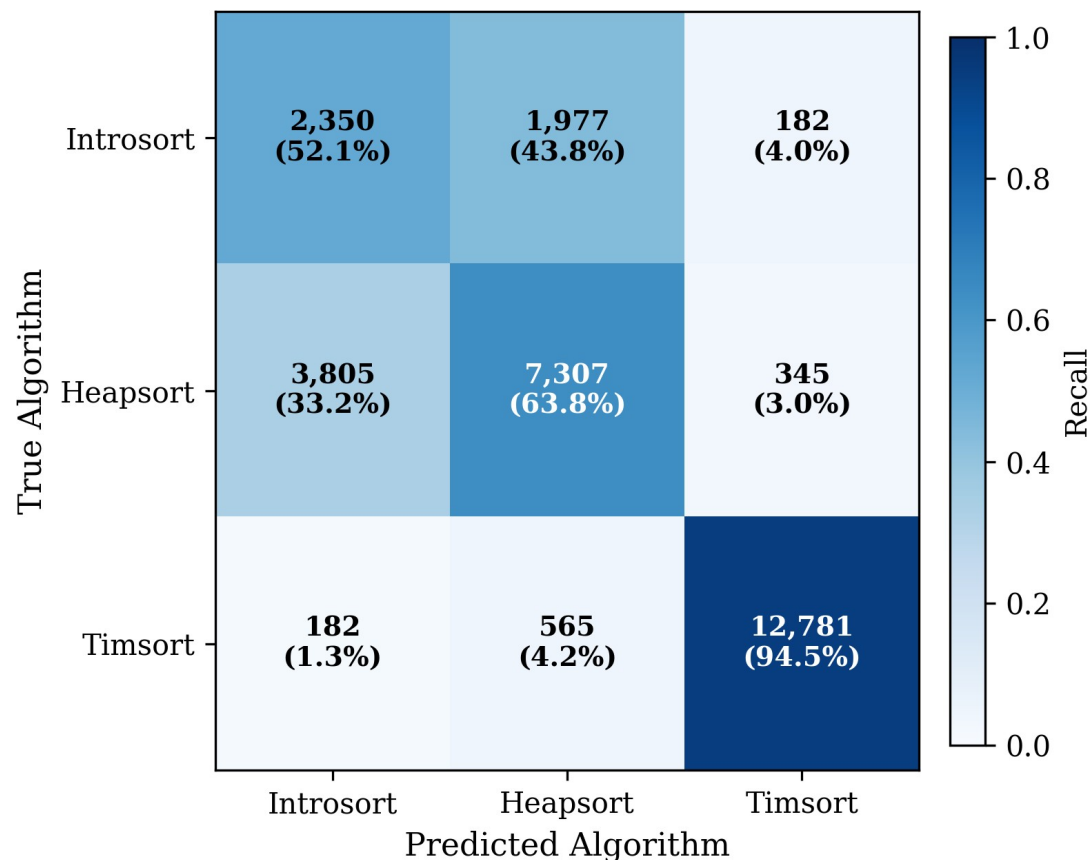
76.1%
label accuracy

93.1%
gap closed

So I do not defend the model as a perfect classifier. I defend its regret.

The hardest boundary is between close algorithms

The confusion matrix shows strong timsort behavior and weaker separation where runtimes are close.



94.5%
timsort recall

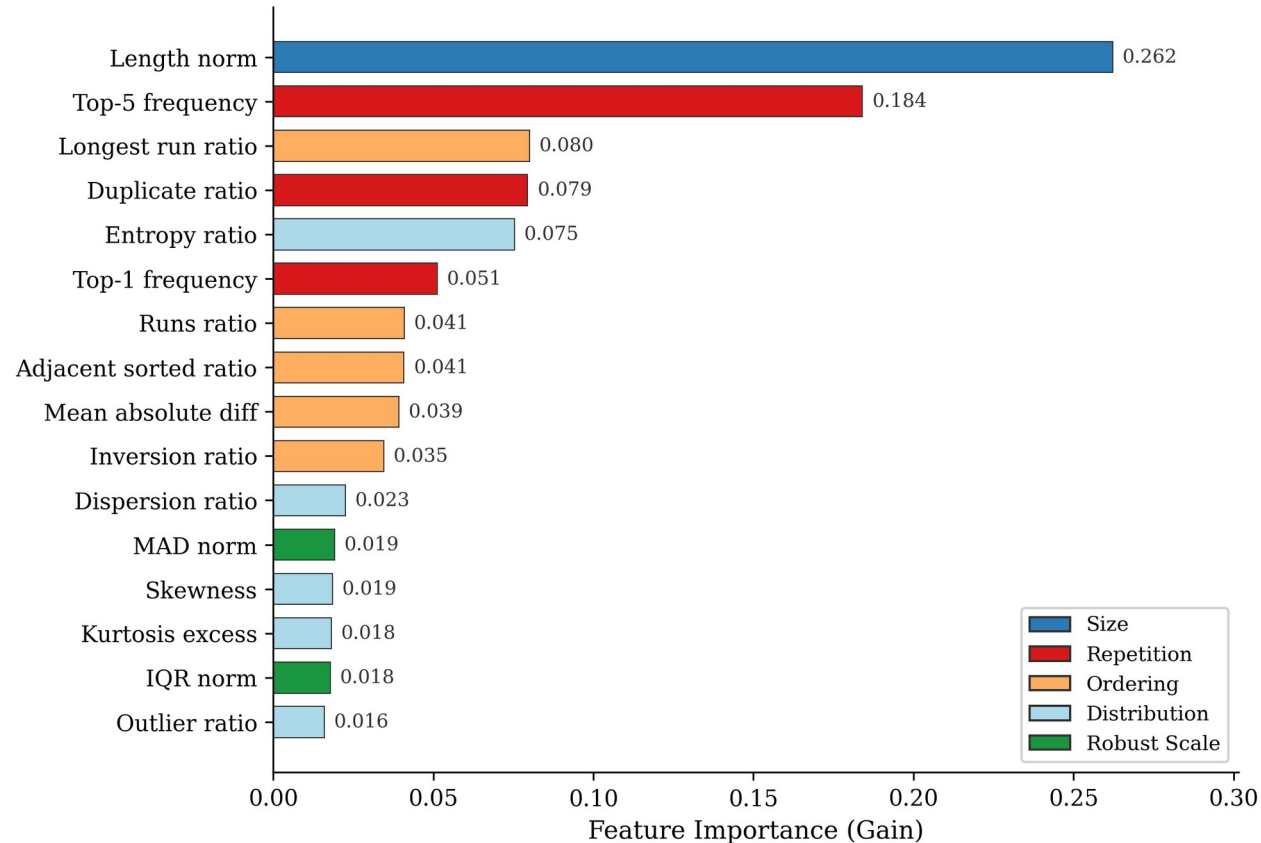
Interpretation

Timsort is easier when order signals are present.
Intro and heap can be much closer.

This is a limitation, but regret tells us many of these errors are low cost.

The model uses structural signals, not magic

Feature importance links the learned choices back to the array properties we extracted.



Order signals

runs, inversions, monotonicity

Value distribution

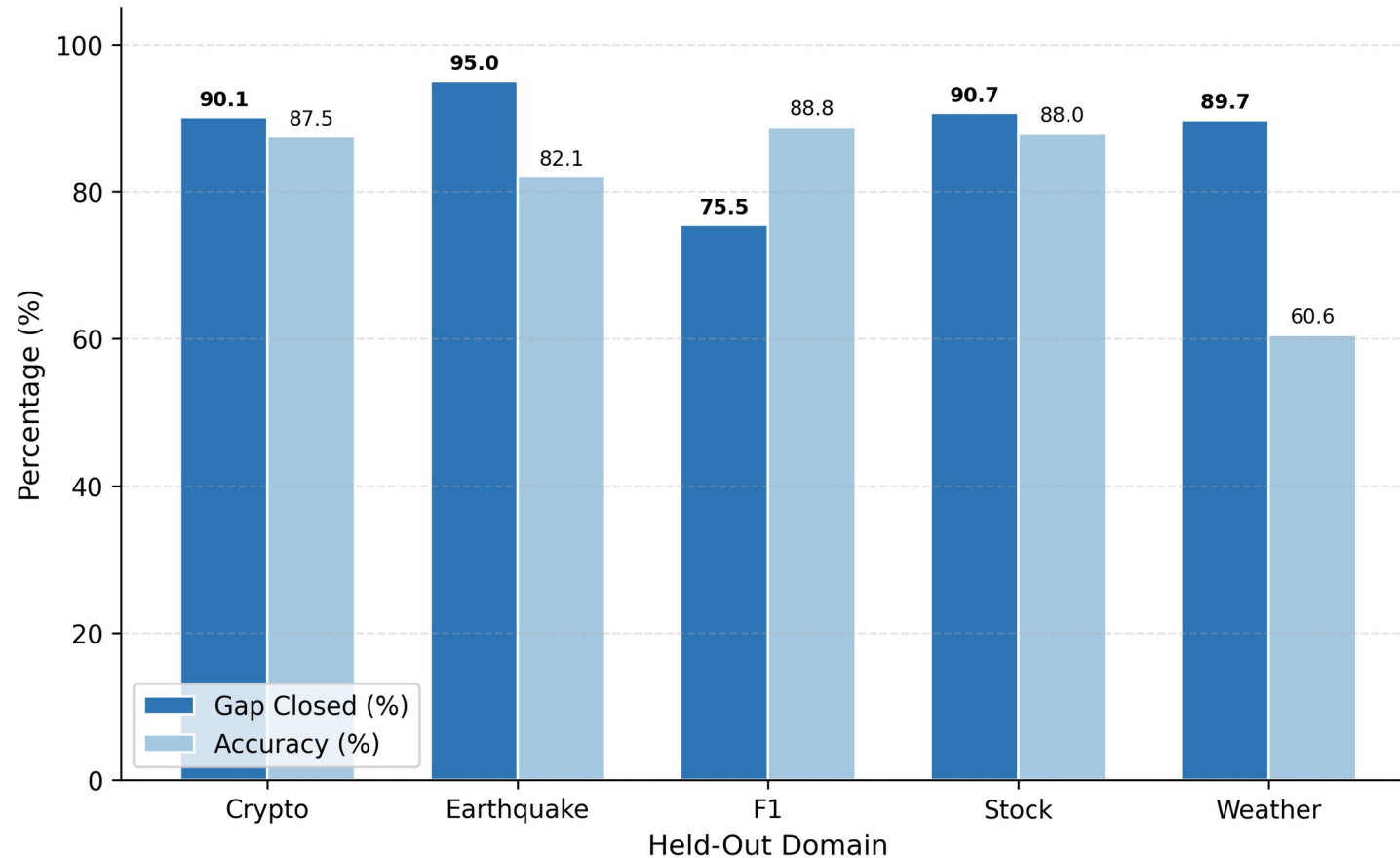
duplicates, range, entropy

Size context

array length and scale

The selector still transfers across held-out domains

Domain holdout is stricter than a random split because the test domain was unseen during training.



79.7%
weighted gap closed

Important caveat

F1 is the hardest held-out domain, which motivates the separate F1 specialization.

Strict checks changed what I was willing to claim

Several experiments were useful because they reduced overclaiming.

v6 source-aware

Accuracy dropped to 71.2%.
This made leakage checks
part of the defense.

v7 regret-aware

Optimizing the wrong signal
reduced gap closed to
78.1%.

v8 cascade

Stage 2 stayed weak
when intro and heap
were too close.

So the final claim stays focused: v5 works well under the submitted evaluation, with known limits.

What I claim, and what I do not claim

| The contribution is strong, but it is not a universal sorting theorem.

I claim

v5 closes most measured runtime
Cheap structural features are used
Limitations are visible in the results

I do not claim

The model is perfect.
F1 specialization is the main result.
LinUCB is validated in this thesis.

The F1 route is a separate specialization: useful to discuss, but not mixed with v5.

Thank you

Questions?

Contribution 1

sorting selection framed as
a measured ML problem

Contribution 2

cheap structural features
across five domains

Contribution 3

93.1% gap closed with
clear limitations

Future work: stronger domain adaptation, validated online selection, and deeper F1-specific evaluation.